

Network Automation — Baby-Steps Guide

A slower, friendlier companion to your CYB/135 Lesson 4 pocket reference

This guide covers the same five tools as your pocket reference (Paramiko, Netmiko, Ansible, NAPALM, Telnet) but starts from zero. Read it top to bottom once, then keep the pocket reference as your fast lookup.

0. The 30-Second Mental Model

Before any code, picture what you're actually replacing:

Without automation: You type a password, log into a router over SSH, type `show version`, read the screen, log out. Repeat that 50 times for 50 routers.

With automation: You write a script that does the typing for you, 50 times, in seconds, and saves the output to a file instead of your screen.

That's it. Everything below is just "how do I get Python to type the SSH stuff for me."

1. Vocabulary First (the pocket reference assumes you already know these)

Term	Plain-English meaning
Device	Any router, switch, or firewall you'd normally SSH into by hand
CLI	Command Line Interface — the text-only screen you type commands into after logging into a device
SSH	A secure, encrypted way to remotely control a device's CLI (replaces old, insecure Telnet)
Library	Pre-written Python code you <code>import</code> so you don't write SSH-handling from scratch
Driver	A class that wraps a library and knows how to talk to one specific <i>kind</i> of thing (a Cisco router, a Linux box, etc.)
Config	The settings stored on a device — what it does, how it's wired, what's allowed
Idempotent	A fancy word for "running it twice doesn't break anything." Turning a light switch ON when it's already ON is idempotent. Flipping it (toggle) is not.
Golden standard	The "correct" config every device is supposed to match (used in audits)

Keep this table open in another tab. Every section below leans on it.

2. The 5-Step Workflow, Slowed Down

Your pocket reference shows this:

Inventory → Connect → Execute → Parse → Act

Here's what each word actually means in practice:

1. **Inventory** — A list of devices and their login info. Usually a simple file like `routers.yml`. Think of it as a spreadsheet: IP address, username, password, vendor.
2. **Connect** — Open a session to one device (SSH is the normal choice today).
3. **Execute** — Send a command, like `show version` or `configure terminal`.
4. **Parse** — The device sends back a wall of text. "Parse" means pulling out just the piece you care about (e.g., the IOS version number) instead of reading the whole wall.
5. **Act** — Do something with what you found: save it to a file, email a report, or push a config change.

Every tool in this guide is just a different way to do steps 2–4. Step 1 (inventory) and step 5 (act) are mostly plain Python you already know.

3. Your First Script — Paramiko, Line by Line

Paramiko is the lowest-level tool: it just knows "SSH," nothing about routers specifically. Good first stop because nothing is hidden from you.

```
import paramiko

class paramiko_driver:
    def __init__(self, hostname, username, password):
        self.hostname = hostname
        self.username = username
        self.password = password
        self.client = paramiko.SSHClient()
        self.client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    def connect(self):
        self.client.connect(
            hostname=self.hostname,
            username=self.username,
            password=self.password
        )

    def send_command(self, command):
        stdin, stdout, stderr = self.client.exec_command(command)
        return stdout.read().decode("utf-8")

    def close(self):
```

```
self.client.close()
```

Walking through it like a beginner would read it:

- `paramiko.SSHClient()` — "create a blank SSH remote control."
- `set_missing_host_key_policy(...)` — "don't panic if this is the first time we've connected to this device" (fine for labs; be more careful in production).
- `connect()` — "actually dial in" using the hostname/username/password you stored.
- `exec_command(command)` — "type this command and press enter."
- `stdout.read().decode("utf-8")` — the device's reply comes back as raw bytes; `.decode("utf-8")` turns it into a normal readable string.

Using it:

```
ssh = paramiko_driver("192.168.1.10", "admin", "secret")
ssh.connect()
print(ssh.send_command("uptime"))
ssh.close()
```

What you should notice: four separate steps — connect, send, read, close — every single time. That's the "verbose" trade-off your pocket reference warns about.

4. The Easier Version — Netmiko

Netmiko is Paramiko, but it already knows things like "Cisco routers print a `--More--` prompt for long output, please page through it for me." That's the whole pitch.

```
from netmiko import ConnectHandler

class netmiko_driver:
    def __init__(self, device_params):
        self.device_params = device_params
        self.connection = None

    def connect(self):
        self.connection = ConnectHandler(**self.device_params)

    def get_version(self):
        return self.connection.send_command("show version")

    def disconnect(self):
        if self.connection:
            self.connection.disconnect()
```

The only new idea here is `device_params` — a plain dictionary that tells Netmiko what *kind* of device it's talking to:

```
ios_device = {
    "device_type": "cisco_ios",  # <- this is the magic ingredient Paramiko
    "host": "192.168.1.1",
    "username": "admin",
    "password": "secret",
    # ... other params ...
}
```

```

}

net = netmiko_driver(ios_device)
net.connect()
print(net.get_version())
net.disconnect()

```

Rule of thumb: if the thing you're connecting to is a router/switch/firewall, use Netmiko. If it's a regular Linux server (or something Netmiko doesn't recognize), use Paramiko.

5. "I Have 50 Devices" — Ansible

Paramiko and Netmiko connect to *one device per script run* (you'd loop yourself for more). Ansible flips the approach: you describe the *end state* you want, and it figures out how to get every device there.

Two files instead of one:

```

# inventory.ini - who are we talking to?
[cisco_routers]
192.168.1.1 ansible_network_os=ios ansible_user=admin ansible_password=secret

# backup.yml - what do we want done?
- name: Backup router configs
  hosts: cisco_routers
  gather_facts: no
  tasks:
    - name: Run show running-config
      ios_command:
        commands: show running-config
      register: config_output
    - name: Save to file
      copy:
        content: "{{ config_output.stdout[0] }}"
        dest: "backups/{{ inventory_hostname }}.cfg"

```

Read it like a recipe, not code: "On every host in the `cisco_routers` group, run `show running-config`, then save the result to a file named after that device."

You *can* kick this off from Python if you want it inside a bigger script:

```

import ansible_runner

class ansible_runner_wrapper:
    def __init__(self, playbook_path, inventory_path):
        self.playbook_path = playbook_path
        self.inventory_path = inventory_path

    def run(self):
        result = ansible_runner.run(
            private_data_dir=".",
            playbook=self.playbook_path,
            inventory=self.inventory_path
        )
        return result.status, result.rc

```

Why this matters for security work: Ansible playbooks are *idempotent* by design — running the

same playbook 100 times produces the same end state every time. That's exactly what you want for golden-config enforcement.

6. "My Devices Are Different Brands" — NAPALM

Cisco, Juniper, and Arista each have their own CLI syntax. NAPALM hides those differences behind one shared Python API, so the *same code* works no matter the vendor.

```
from napalm import get_network_driver

class napalm_driver:
    def __init__(self, vendor, hostname, username, password):
        driver = get_network_driver(vendor)  # <- picks the right translator
        self.device = driver(hostname, username, password)

    def connect(self):
        self.device.open()

    def load_and_merge(self, config_file):
        self.device.load_merge_candidate(filename=config_file)
        return self.device.compare_config()  # shows you the diff BEFORE
applying

    def commit(self):
        self.device.commit_config()

    def rollback(self):
        self.device.discard_config()
```

The standout safety feature: `compare_config()` shows you a *preview* of the change before it's live. That's the "review the diff before commit" pattern your pocket reference's example follows:

```
napalm = napalm_driver("ios", "192.168.1.1", "admin", "secret")
napalm.connect()
diff = napalm.load_and_merge("snippets/ntp.cfg")
print(diff)  # look before you leap
if "ntp server" in diff:
    napalm.commit()
else:
    napalm.rollback()
```

7. Telnet — Why You'll Almost Never Use It

Telnet is the same idea as SSH (open a remote CLI session) but invented decades earlier, before anyone worried about encryption. Every password and command travels in **plain, readable text** — anyone sniffing the network traffic can read your login.

You'll only ever touch it for:

- A genuinely ancient piece of lab gear that has no SSH option.
- A controlled lab/CTF environment where that's the point of the exercise.

If you ever see Telnet in a production environment during this course's "auditing" exercises, that's a finding, not a tool you reach for.

8. Paramiko vs Netmiko, Simplified

Question	Answer
Connecting to a Linux server?	Paramiko
Connecting to a router/switch/firewall?	Netmiko
Need it to handle "press space to see more" prompts for you?	Netmiko
Need full manual control over every byte of the SSH session?	Paramiko
Need built-in <code>enable()</code> for privileged mode?	Netmiko

9. Best Practices, in Plain English

Pocket-reference rule	What it actually means day-to-day
Use SSH, not Telnet	Don't let passwords travel in plain text
Store secrets in vaults	Never hardcode <code>password = "secret"</code> in a script you commit to GitHub
Back up before change	Run <code>show running-config</code> and save it <i>before</i> you touch anything
Test in a lab first	GNS3 / EVE-NG / Cisco CML before real gear
Make changes idempotent	"Run it twice, nothing breaks" — check before adding, don't blindly append
Parse with templates	Use TextFSM/Jinja2 instead of manually slicing strings with <code>.split()</code>
Version-control everything	Your inventory, playbooks, and scripts live in Git, not just on your laptop
Log and audit	If a script changed a device, there should be a record of who/what/when

10. One-Page Cheat Sheet (print this part)

Need to...	Reach for...
Raw SSH to a Linux server	Paramiko
SSH to a router/switch/firewall	Netmiko
Push the same config to 50 devices	Ansible
Same code across Cisco/Juniper/etc	NAPALM
Only option is plain-text legacy	Telnet (lab only)
Turn CLI text into structured data	TextFSM + Netmiko

The 5-step workflow:

Inventory → Connect → Execute → Parse → Act
(devices.yml) (SSH/API) (show/config) (structured) (save/report/rollback)

11. How This Maps to Your Lesson 4 Objectives

- **Understand network automation and its working** → Sections 0–2
 - **Describe the types of network automation** → see your pocket reference §3 (config mgmt, monitoring, auditing, provisioning, event-driven) — this guide builds the foundation those types sit on top of
 - **Understand the benefits** → Section 9 connects each "best practice" to a real benefit (speed, consistency, auditability)
 - **Describe Paramiko, Ansible, Telnet, Netmiko, NAPALM** → Sections 3–7
 - **Difference between Paramiko and Netmiko** → Section 8
 - **Best practices and future** → Section 9 + your pocket reference §8 (intent-based networking, API-first devices, GitOps)
-

Next step if you want it: this could become a `ch-network-automation` lesson inside `py-bite` with the same five drivers as interactive exercises — let me know if you want that scaffolded out.